如何用 AI Agent

从 *Architect* → *Kernel*　*Author* → *Ship* 的全流程实战

SZY

2026 年 3 月 24 日

目录

I 天真期：先跑起来再说 **3**

1 从零开始：一个人和一个 AI **4**

1.1 起念 4

1.2 一路狂飙：GDT、IDT、键盘 4

1.3 已经埋下的雷 5

1.4 第一本书的野心 6

2 失控：当代码开始反噬 **7**

2.1 PMM 噩梦 7

2.2 注释荒漠 7

2.3 书稿的裂缝 8

2.3.1 TikZ 图的灾难 8

2.3.2 frame=single 的惨案 8

2.3.3 代码与书稿不同步 9

2.4 痛点总结 9

II 第一次收敛：引入结构 **10**

3 里程碑拯救了我 **11**

3.1 刹车 11

3.2 为什么 Roadmap 有效 12

3.2.1 问题 1：我不知道下一步做什么 12

3.2.2 问题 2：AI 不知道项目全貌 12

3.2.3 问题 3：不知道什么时候算「做完了」 12

3.3 第一次让 AI 帮我规划 12

3.4 Changelog：对抗遗忘 13

3.5 收敛的感觉 14

4 发现模式：可插拔接口 **15**
4.1 第一次重构 15
4.2 模式不断重复 16
4.3 AI 不会自动套用模式 16
4.4 测试作为接口契约 17
4.5 此时的开发流程 17

5 AI 记不住的规则 **19**
5.1 重复说教的疲惫 19
5.2 问题的本质 20
5.3 第一次尝试：copilot-instructions.md 20
5.4 新的问题：全塞在一起太臃肿 21

III Agent 化：把规则变成配置 **22**

6 Agent 诞生 **23**
6.1 拆分 23
6.2 三个 Agent 文件 + 一个 Prompt 23
6.2.1 Architect：只读的架构师 23
6.2.2 Kernel：受约束的开发者 24
6.2.3 Author：限域的作家 24
6.2.4 Ship：一次性的检查员 25
6.3 copilot-instructions.md 瘦身 25
6.4 效果验收 25

7 Design Spec：Agent 之间的契约 **27**
7.1 Architect 的输出需要标准化 27
7.2 Design Spec 模板的演化 27
7.2.1 v1：最小可用 27
7.2.2 v2：加了 Verification 27
7.2.3 v3：拆分 Kernel Plan 和 Book Plan 28
7.3 实例：Stage D-1 TSS + Ring 3 28
7.4 Design Spec 的核心价值 30

8 终于能并行了 **31**
8.1 解锁并行的那一刻 31
8.2 并行的安全保障 32
8.2.1 第一层：文件边界 32

8.2.2 第二层：Design Spec 是快照 32
8.2.3 第三层：偏离报告 32
8.3 实际的工作节奏 32
8.4 对比：三个阶段的效率 33

IV 持续演进 34

9 Agent 也需要成长 35
9.1 不是一次性的 35
9.2 案例 1：TikZ 图内联问题 35
9.3 案例 2：compile_commands.json 遗漏 36
9.4 案例 3：Architect 的前瞻性不足 36
9.5 演化时间线 36
9.6 反馈循环 37
9.7 什么时候该加规则，什么时候不该 37

10 复盘：可迁移的经验 39
10.1 回顾旅程 39
10.2 七条可迁移的经验 40
10.2.1 经验 1：先裸奔，后穿衣 40
10.2.2 经验 2：战略人定，战术 AI 定 41
10.2.3 经验 3：工具权限是硬边界 41
10.2.4 经验 4：中间产物标准化 41
10.2.5 经验 5：规则从反馈中长出来 41
10.2.6 经验 6：并行需要不重叠的写入范围 41
10.2.7 经验 7：Roadmap 是最高杠杆率的上下文 42
10.3 这个方法论的局限 42
10.4 下一步 42

V 附录 44

A Agent 配置文件完整参考 45
A.1 Architect Agent 45
A.2 Kernel Agent 46
A.3 Author Agent 46
A.4 Ship Prompt 47
A.5 copilot-instructions.md（全局共享） 48

B 项目时间线 **49**
B.1 完整时间线 49
B.2 演化阶段总结 50

Part I

天真期：先跑起来再说

Chapter 1

从零开始：一个人和一个 AI

1.1 起念

2026 年 3 月的一个周六，我打开终端，敲下了这几个命令：

```
mkdir my_microkernel
cd my_microkernel
mkdir -p src/{boot,kernel,include}
```

目标很简单也很疯狂：**写一个 x86 微内核操作系统**，从 Multiboot2 引导到最终能跑 GCC 和 Vim。

我的全部装备：一台装了 Ubuntu 的电脑、QEMU 模拟器、一个 AI 编码助手（VS Code 里的 GitHub Copilot），以及从大学操作系统课上残存的模糊记忆。

没有团队。没有导师。没有预先的架构设计。

我打开 Copilot Chat，输入了第一句话：

> **Human**
>
> 帮我写一个 Multiboot2 引导的最小内核，NASM 汇编入口，调用 C 语言的 kernel_main，向 VGA 文本模式缓冲区输出"Hello Kernel"。

30 秒后，AI 给了我一份 `boot.asm` 和一份 `kmain.c`。我把它们存下来，写了个 Makefile，`make iso`，扔进 QEMU——

屏幕上出现了 `Hello Kernel`。

这太爽了。

此刻我犯了一个所有人都会犯的错误：我以为接下来会一直这么顺利。

1.2 一路狂飙：GDT、IDT、键盘

接下来几个小时，我沉浸在一种令人上瘾的节奏里：

1. 问 AI：「帮我实现 GDT」
2. 拿到代码，塞进项目
3. 编译、运行、看到结果
4. 问 AI：「帮我实现 IDT」
5. 重复

速度快得不可思议。GDT（全局描述符表）的平坦段模型、IDT（中断描述符表）的异常处理、8259 PIC 的初始化、PS/2 键盘驱动——一个下午搞定了正常情况下需要一两周的工作量。

到了晚上，我有了一个能敲键盘交互的 Shell。

```
$ Terminal

> help
Available commands: help, cls, shutdown
> cls
```

这个阶段，我和 AI 之间没有任何结构——不存在设计文档、没有里程碑、没有代码规范。就是「我要什么就问什么，AI 给什么我就用什么」。

用一个词概括：**即兴**。

即兴的好处是启动快、心理负担低、成就感爆棚。

即兴的坏处……我还不知道，但很快就会知道了。

1.3 已经埋下的雷

当时的我不知道，但回过头来看，前四个 Stage 的代码里已经埋了好几颗雷：

1. **GDT 只有 3 项**。AI 按照我当时的需求（平坦代码 seg + 数据 seg）生成了 3 项 GDT。后来做特权级切换时，要加 User Code、User Data、TSS——不得不回头改。
2. **IDT 的 `regs_t` 不完整**。中断帧结构体没有 `user_esp` 和 `user_ss` 字段。跨特权级中断时 CPU 会额外压栈两个值，但这时候的代码根本没考虑。
3. **零注释**。一行 `entry.access = 0x9A` 里压缩了 8 个位域的含义。我当时看得懂（因为刚写完），但一周后就忘了。
4. **没有测试**。代码能跑就算成功。后来改了 GDT 结构后引入回归 Bug，花了两小时才发现。

这些问题现在说起来很明显，但在当时，沉浸在「问 AI → 代码 → 能跑」的多巴胺循环里，我完全没有意识到自己正在堆积技术债务。

思考 1.1

现在暂停一下，想想你自己的经历：你有没有过一段「AI 辅助狂飙」的体验？那个阶段最终是怎么结束的——是你主动踩刹车，还是项目本身撞墙了？

1.4 第一本书的野心

在做完 Shell 之后，我产生了一个额外的想法：**我要同时写一本配套的教材**。

理由很充分：

- 写书能帮助我自己理解代码（「如果你不能向别人解释，说明你自己没搞懂」）
- 这个项目本身就是教学导向的，有书才完整
- AI 也能帮我写 LaTeX 啊！

于是我又打开 Copilot Chat：

Human

帮我写一个 LaTeX 书稿的框架，用 XeLaTeX + ctex 中文支持，O'Reilly 风格封面，每个 Stage 对应一章。

AI 给了我 `main.tex` 和 `preamble.tex`。我花了点时间调整样式（代码高亮配色、终端输出的深色背景框），然后开始**一边写代码一边写书**。

这个决定让项目的复杂度直接翻倍——但我还没意识到。此时我的开发流程是：

1. 让 AI 写一段内核代码
2. 测试通过
3. 让 AI 写对应的 LaTeX 章节
4. 编译 PDF，检查排版

一切看起来很好。但这是一个**严格串行**的流程——代码写完才能写书，书写完才能下一个 Stage。而且整个过程中，AI 不记得之前的上下文——每次对话我都要重新描述项目背景。

很快，我就会撞墙了。

Chapter 2

失控：当代码开始反噬

2.1 PMM 噩梦

里程碑 B 的第一个 Stage——物理内存管理器（PMM）——是一切崩塌的开始。

我对 AI 说：

> **Human**
>
> 帮我实现物理内存管理。从 Multiboot2 memory map 解析可用区域，用 bitmap 跟踪页帧，支持 alloc_page 和 free_page。

AI 给了我大约 300 行 C 代码。编译通过，运行通过，能分配页帧。我很满意。

一天后，我仔细看了一遍代码，发现了这一行：

```
static uint8_t *bitmap = (uint8_t *)0x100000; // Bad!
```

位图的起始地址是硬编码的。

这意味着：如果物理内存布局不是 AI 假设的那样，内核会直接覆写有用的数据。在真实硬件上这是一个必现的内存损坏 Bug。

我让 AI 修。AI 修了——但「修」的方式是把 `0x100000` 换成了 `0x200000`。还是硬编码。

我又纠正了一轮，这次明确说「从 Multiboot2 mmap 动态计算」。第三轮才拿到正确的代码。

三轮对话解决一个问题。而且如果我没有仔细审查，这个 Bug 会一直待在那里。

2.2 注释荒漠

PMM 的代码审查还暴露了另一个问题：AI 生成的底层代码**几乎没有注释**。

下面这段页表操作代码，你能一眼看出每一位的含义吗？

```
page_table[pt_idx] = (phys_addr & 0xFFFFF000) | 0x03;
```

`0x03` 是什么？Present + Read/Write。那 User 位呢？不设。为什么？因为这是内核页。但代码里没有任何解释。

一周后写 VMM（虚拟内存管理）时，我忘了这些位域的含义，又花了 20 分钟对照 Intel 手册。两周后做用户态页表映射，又一次。

同一个知识点我查了三遍。

问题不在于 AI 写的代码不对——代码能跑。问题在于 **AI 生成代码的默认行为是「最小可用」，不是「最大可维护」**。它不会主动加注释，除非你要求它。但你要求一次，下次它又忘了。

2.3 书稿的裂缝

与此同时，书稿开始出问题。

2.3.1 TikZ 图的灾难

我让 AI 画一些内存布局的 TikZ 图（内存位图、页表结构、地址分割）。AI 确实给了——但图里的文字和格子**重叠**了。bitcell 宽度太窄，长文本溢出；节点间距不够，箭头穿过文字。

我修了一张 →AI 忘了 → 第二张又重叠。修了第二张 → 第三张又来。

总共 60 多张 TikZ 图，我返工了 15 张。

2.3.2 frame=single 的惨案

AI 生成的 `lstlisting` 代码框用的是 `frame=single`（四面画框）。看起来还不错——**直到代码跨页**。

LaTeX 的 `frame=single` 在跨页时不会封闭矩形框的底部和顶部，导致 PDF 里出现「断开的方框」。这不是个别现象——整本书里到处都是。

踩坑警告

教学书籍的代码块几乎必然跨页。`frame=single` 在这种场景下是一个已知的烂选择，但 AI 每次都默认用它。这就是「AI 不会犯 Bug，但会反复做出次优选择」的典型案例。

我把它改成 `frame=l`（只画左侧竖线），跨页问题立刻消失。但这时候已经有 98 处代码块用了 `frame=single`——我不得不全局替换。

2.3.3 代码与书稿不同步

最致命的问题：当我修改了内核代码（比如给 PMM 加了多区域支持），**书稿里引用的代码还是旧版本**。

因为代码和书稿是两次独立的对话，AI 不知道代码已经改了。我在书稿里写的是 v1 的 PMM 接口，但 `src/` 里已经是 v2 了。

到里程碑 B 结束时，代码和书稿之间已经有了多处不一致，我花了大半天时间手动对齐。

2.4 痛点总结

停下来盘点一下，此时项目面临的所有问题：

问题	表现	根因
硬编码	物理地址写死，换环境就崩	AI 默认「最小可用」
注释缺失	位域含义反复查手册	没有约束 AI 必须注释
TikZ 重叠	15 张图需要返工	AI 不知道我的样式参数
frame=single	跨页方框断裂	AI 的默认选择是次优的
代码/书稿不同步	书里的代码是旧版	串行流程 + 无版本追踪
每次都要重复背景	对话效率低	AI 不记得项目上下文
没有测试	改 GDT 后引入回归	没有验证环节
没有计划	写到哪算哪	没有里程碑

表 2.1: 里程碑 A-B 期间的痛点清单

看着这张表，一个事实变得很清楚：**问题不在 AI 的能力，而在于我使用 AI 的方式**。

我把 AI 当成了「一个什么都会的实习生」——要什么给什么，但每次都要从头解释背景，不会自己管质量，不会自己测试，不记得之前的决策。

这不是 AI 的问题。这是**管理**的问题。

我需要的不是更强的 AI，而是一套**工作流**——让 AI 在规则之内工作，让质量标准内化到流程里，让重复的事情不需要每次说。

思考 2.1

重新看一遍上面的痛点表。你自己的 AI 辅助开发中，遇到了哪几条？你现在是怎么处理的——每次手动纠正，还是已经找到了系统性的解决方案？

Part II

第一次收敛：引入结构

Chapter 3

里程碑拯救了我

3.1 刹车

里程碑 B 结束后，我做了一个关键决定：**停下来，先写 roadmap**。

不是因为我很有纪律——是因为我已经不知道接下来该做什么了。VMM 要分页，分页要页表，页表要物理内存，但我已经有了 PMM，下一步是先做虚拟内存还是先做 High-Half Kernel?

当你连下一步是什么都想不清楚的时候，说明你需要一张地图。

我打开一个空文件 `docs/roadmap.md`，开始写：

```
# SZY-KERNEL 项目愿景与路线图

## 终极目标
构建一个 x86 微内核操作系统，最终能：
- 自举移植 GCC 编译器
- 运行 Vim 文本编辑器
- 支持 TCP/IP 网络访问

## 里程碑 A：裸机基础设施 [已完成]
## 里程碑 B：物理内存管理 [已完成]
## 里程碑 C：高级内存   [进行中]
## 里程碑 D：进程与用户态 [计划]
## 里程碑 E：文件系统   [计划]
...
```

这个文件改变了一切。

3.2 为什么 Roadmap 有效

Roadmap 解决了三个问题：

3.2.1 问题 1：我不知道下一步做什么

之前：每次打开项目都要想「今天做什么」。选择焦虑导致我经常挑简单的（调个配色）而不是重要的（实现 VMM）。

之后：Roadmap 里的 Stage 是有序的。打开文件，找到第一个非 的 Stage，开工。

3.2.2 问题 2：AI 不知道项目全貌

之前：每次对话都要重新解释「这是一个 OS 项目，目前做到了 XXX，下一步要做 YYY」。

之后：把 `docs/roadmap.md` 丢给 AI，它 30 秒就能理解项目的完整上下文——过去做了什么、现在在做什么、将来要做什么。

Prompt 技巧

Roadmap 是你能给 AI 的**最高杠杆率的上下文**。一份 100 行的 roadmap，比 1000 行的对话历史更有信息量。因为 roadmap 是结构化的、有序的、去噪的。

3.2.3 问题 3：不知道什么时候算「做完了」

之前：一个功能可以无限打磨。PMM 能跑就算完了？还是要加多区域支持？还是要写测试？

之后：每个 Stage 有明确的「内容」和「状态」。完成了标 ，没完成标空。完成标准是 Stage 描述里列出的东西——不多做也不少做。

3.3 第一次让 AI 帮我规划

有了 roadmap 的框架之后，我做了一个有意思的实验：让 AI 来细化里程碑。

Human

阅读 docs/roadmap.md，帮我把里程碑 D（进程与用户态）从目前的 4 个粗粒度 Stage 细化为递进的子步骤。参考里程碑 B/C 的"概念 → 接口 → 后端"模式。

AI 读完 roadmap 和已有代码后，把里程碑 D 从 4 个 Stage 扩展到了 12 个，按 / / / / / 子阶段分组。这个结果非常好——因为 AI 能看到整个 roadmap 的模式并复用。

但更重要的是这个过程揭示的**方法论**：

1. **人定目标**：终极目标（跑 GCC/Vim/TCP）是我定的，AI 没有资格替你决定做什么项目
2. **人划里程碑**：粗粒度的 A/B/C/D/E 划分是我做的，基于我对 OS 知识体系的理解
3. **AI 细化 Stage**：每个里程碑内部的具体步骤和排序，AI 做得比我好——它能看到更多的模式和依赖

经验教训

战略人定，战术 AI 定。你决定去哪里，AI 帮你选最佳路径。反过来不行——如果让 AI 决定你要做什么项目、做几个里程碑，结果会是一份看起来很漂亮但跟你的能力和兴趣完全对不上的计划。

3.4 Changelog：对抗遗忘

Roadmap 告诉我「将来要做什么」，但还需要一个东西告诉我「过去做了什么」。于是我开始维护 `docs/changelog.md`——每次完成一个 Stage，记录：

- 新增/修改了哪些文件
- 关键的技术决策和理由
- 验证方法和结果

这个文件后来成为了整个项目最有价值的文档之一。原因是：

1. **AI 的上下文窗口**比你的记忆更可靠。把 changelog 丢给 AI，它能精确回忆「上次改 vmm.c 是因为什么」。
2. **写书时的素材**。书稿的很多段落直接从 changelog 里改写的——因为 changelog 记录了每个决策的 WHY。
3. **Debug 的线索**。当某个功能突然不工作时，倒着翻 changelog 找「哪次改动可能破坏了它」。

3.5 收敛的感觉

有了 Roadmap + Changelog 之后，项目从「漫无目的地写代码」变成了：

```
1. 打开 roadmap.md，找到当前 Stage
2. 告诉 AI "实现 Stage C-2: 内核堆 first-fit"
3. AI 读 roadmap + 已有代码，实现
4. 测试通过
5. changelog.md 追加记录
6. roadmap.md 标
7. 下一个 Stage
```

这就是「收敛」的感觉——你知道自己在哪里、要去哪里、刚才做了什么。不再有选择焦虑，不再有「我是不是在做无用功」的怀疑。

但这只是第一次收敛。新的问题很快会出现：代码质量的规范化、AI 反复犯同一个错误的问题、以及书稿和代码无法并行的瓶颈。

这些问题将催生下一个进化——可插拔接口模式，和让 AI 「记住规则」的需求。

练习 3.1

为你的项目写一份 roadmap。规则：

1. 用一段话定义终极目标
2. 划分 3-5 个里程碑（先粗后细）
3. 每个里程碑用一张表列出 Stage（名称 + 内容 + 状态）
4. 把这份 roadmap 给 AI 看，让它帮你细化其中一个里程碑
5. 对比 AI 的细化和你自己想的——哪些是你没想到的？

Chapter 4

发现模式：可插拔接口

4.1 第一次重构

里程碑 C 是内存管理的「高级篇」——堆分配器（kmalloc）和虚拟地址空间管理（VMA）。在做堆分配器时，我想到了一个问题：

PMM 已经有了 bitmap 后端。但我知道 buddy system 更好（适合大块分配、合并效率高）。如果以后要换成 buddy，所有调用 PMM 的地方都要改吗？

答案是：**不用，如果你有一个接口层**。

于是我做了进入这个项目以来第一次真正的**架构性重构**：

1. 从 bitmap 实现中抽取出公共 API（`pmm_alloc_page`、`pmm_free_page`）
2. 定义一个函数指针表 `pmm_ops_t`，包含这些 API 的签名
3. 把 bitmap 的实现改成「填充 `pmm_ops_t`」的形式
4. 写一个 dispatch 层，通过全局指针转发调用
5. 启动时注册具体的后端

代码长这样：

```
// 接口定义
typedef struct {
    const char *name;
    void (*init)(uint32_t mem_size);
    uint32_t (*alloc_page)(void);
    void (*free_page)(uint32_t addr);
} pmm_ops_t;

// Dispatch 层
```

```
static const pmm_ops_t *g_pmm_ops = NULL;

uint32_t pmm_alloc_page(void) {
    return g_pmm_ops->alloc_page();
}
```

重构完成后，我用同一个接口实现了 buddy 后端。切换只需要改一行 `kconfig.h`：

```
#define KCONFIG_PMM_BACKEND 1 // 0=bitmap, 1=buddy
```

编译、运行、pmm test 全过。两个后端在完全不同的数据结构上实现了完全相同的接口。

4.2 模式不断重复

做完 PMM 之后，堆分配器（kmalloc）也面临同样的问题：first-fit 还是 slab?

答案显而易见：**再搞一个 `heap_ops_t`**。代码结构跟 PMM 一模一样——接口、dispatch、后端、kconfig。

然后是 VMA。sorted-array 还是红黑树还是 maple tree? `vma_ops_t`。

到里程碑 C 结束时，我有了三个可插拔接口，总共 7 个后端实现。所有后端通过相同的测试套件。

接口	后端数量	后端
`pmm_ops_t`	2	bitmap, buddy
`heap_ops_t`	2	first-fit, slab
`vma_ops_t`	3	sorted-array, rbtree, maple-tree

表 4.1: 里程碑 C 结束时的可插拔接口

4.3 AI 不会自动套用模式

这里有一个关键洞察：**虽然我发现了这个模式，但 AI 不会自动记住**。

每次开始一个新的子系统，我都需要重新对 AI 说：

Human

参考 pmm_ops_t 的设计模式——函数指针表 + dispatch 层 + 后端实现 + kconfig 选择——给 VMA 也搞一套。

如果我忘了说「参考 pmm_ops_t」，AI 可能就直接写一个没有接口层的 monolithic 实现。然后我又要花时间重构。

这个「每次都要提醒」的问题变得越来越烦人。我在 changelog 里反复写同样的话：「遵循 `xxx_ops_t` 可插拔接口模式」。同一个模式，同一段描述，复制粘贴了至少五次。

经验教训

当你发现自己在对话里反复说同一段话时，说明这段话**应该被固化成配置**。这是第五章（Agent 诞生）的直接前因。

4.4 测试作为接口契约

另一个重要发现是：**测试用例是跟接口绑定的，不是跟实现绑定的**。

PMM 的 5 项测试（alloc/free 往返、页对齐、唯一性、批量压力、写读验证）对 bitmap 和 buddy 后端都适用。VMA 的 9 项测试对 sorted-array、rbtree、maple-tree 三个后端都适用。

这意味着每次实现新后端时，验证是自动化的——跑一遍 `xxx test`，全过就行。不需要为每个后端单独写测试。

我把测试命令集成到了 Shell 里（`pmm test`、`heap test`、`vma test`），可以在 QEMU 运行时直接执行。后来还写了 `scripts/test.sh`，用 QEMU 的 `-serial stdio` 做自动化回归。

这套测试框架在后续开发中救了我无数次——每次改了一个子系统，跑一遍全部测试，确保没有回归。

4.5 此时的开发流程

到里程碑 C 结束时，我的开发流程已经从最初的「即兴」进化到了：

1. 打开 `roadmap.md`，确定当前 Stage
2. 对 AI 说：「实现 Stage C-5: VMA 红黑树后端。参考 `vma_ops_t` 接口和 `vma_sorted_array.c` 的结构。」
3. AI 实现代码
4. 编译 `make iso`
5. QEMU 里跑 `vma test`，全部通过
6. 对 AI 说：「写 ch11-vma.tex 的红黑树后端章节。`frame=l`，`\codefile{}` 标注，TikZ 图放 `figures/ch11/`。」

7. 编译 PDF
8. 更新 changelog + roadmap 状态

比起里程碑 A 时代的「裸奔」，这已经好多了。但仍然有两个痛点：

1. 每次都要在对话里**复述规则**（接口模式、注释规范、TikZ 目录）——AI 没有持久记忆
2. 代码和书稿**必须串行**——先写代码，再写书稿，不能同时进行

解决这两个问题，就需要一个全新的东西：**Agent 配置文件**。

练习 4.1

在你的项目中识别一个「重复出现的结构模式」（比如：每个 API endpoint 都有 handler + validator + test）。写下这个模式的检查清单。然后数一数：你跟 AI 重复描述过这个模式几次？

Chapter 5

AI 记不住的规则

5.1 重复说教的疲惫

到里程碑 C 结束时，我在每次新 Stage 的对话里都要说一大段「开场白」：

Human

帮我实现 Stage D-1: TSS 和 Ring 3。
注意事项：

- 涉及硬件的代码必须加 [WHY]、[CPU STATE]、[BITFIELDS] 注释
- 遵循 xxx_ops_t 可插拔接口模式
- 所有地址不能硬编码
- 页操作必须 4096 对齐
- 结束后跑 make iso 验证

书稿注意事项：

- frame=l，不要 frame=single
- \codefile{} 标注源文件路径
- TikZ 图放 figures/chNN/ 目录
- 不要在章节文件里内联 TikZ 代码

每次。每个 Stage。同样的话。

这不仅浪费时间，更危险的是：当我疲惫或者着急时，**会忘记说其中某一条**。有一次我忘了说「frame=l」，结果 AI 又用了 `frame=single`，生成了一整章的代码框——我

不得不全部改。

5.2 问题的本质

让我把问题说得更精确：

AI 聊天是**无状态的**。每次新对话，AI 不记得上一次我们约定的规则。它不知道这个项目有一个「可插拔接口模式」的设计规范，不知道代码要加三种注释，不知道 TikZ 图不能内联。

我一直在用**人肉记忆**来弥补 AI 的无状态性。每次对话开头复述规则，本质上是我在充当 AI 和项目之间的 "session storage"。

但人的记忆是不可靠的。而且随着规则越来越多——

类别	规则数量
代码注释规范	3 条（WHY / CPU STATE / BITFIELDS）
内存管理准则	2 条（禁止硬编码、对齐要求）
设计模式	5 步（接口 →dispatch→ 后端 → 注册 →kconfig）
LaTeX 代码框	2 条（frame=l、\codefile）
TikZ 管理	4 条（目录规范、命名、内容隔离、引用方式）
构建验证	2 条（make iso、make compdb）
总计	**18 条规则**

表 5.1: 需要在每次对话中传达的规则

18 条规则。每次新对话都要传达全部（或至少相关的子集）。这不是一个可持续的工作方式。

5.3 第一次尝试：copilot-instructions.md

VS Code 的 GitHub Copilot 有一个功能：你可以在项目根目录放一个 `.github/copilot-instruct` 文件，内容会被**自动注入到每次对话的上下文**里。

这就是答案——把规则写成文件，让 AI 自己去读！

我立刻写了第一版 `copilot-instructions.md`，把前面表格里的 18 条规则全塞了进去。然后加上了 Git 工作流、Merge 检查清单、项目文件结构目录树。

```
# 代码规范

## 学习级注释规范（强制）
- [WHY]: 为什么要操作这个寄存器/标志位
- [CPU STATE]: 此操作后 CPU 的特权级...
```

```
- [BITFIELDS]: 对页表项或描述符位定义的逐位解释

## 内存管理准则
- 禁止 Hardcode
- 对齐要求: 4096

## Book / LaTeX 图片管理规范 (强制)
- TikZ 图片必须放在 book/figures/chNN/ 目录下
...
```

效果立竿见影。我不再需要在每次对话里复述规则了。AI 自动读取了 instructions 文件，生成的代码带上了 [WHY] 注释，LaTeX 代码框用了 `frame=l`。

18 条规则从「我的脑子」搬到了「一个文件」里。

5.4 新的问题：全塞在一起太臃肿

很快，`copilot-instructions.md` 膨胀到了 200 行。包括：

- 代码规范（给写代码的场景用）
- LaTeX 规范（给写书的场景用）
- Git 工作流（给提交的场景用）
- 项目文件结构（给所有场景用）

问题来了：当我让 AI 写代码时，它的 context window 里塞了一堆 LaTeX 规范——**这些信息是噪音**。反过来，写书时，代码注释规范也是噪音。

更严重的是，一个超长的 instructions 文件导致 AI 的注意力被分散。规则越多，每条规则被遵守的概率越低——不是因为 AI 故意违反，而是因为 context 太长，部分规则被「注意力稀释」了。

我需要一种方法，让**不同场景加载不同的规则**。

这就是 Agent 配置文件的由来——下一章的故事。

踩坑警告

Anti-pattern: “把所有规则写在一个文件里”。这是你从 0 条规则进化到 “有规则” 时的第一站，但不应该是终点。当规则超过 20 条时，就该考虑按角色拆分了。

思考 5.1

你的项目有没有类似的 “配置文件膨胀” 现象？比如一个 `.eslintrc` 或 `Makefile` 变得超长，不同的关注点混在一起？你是怎么解决的？

Part III

Agent 化：把规则变成配置

Chapter 6

Agent 诞生

6.1 拆分

解决方案出奇地简单：**把一个大文件拆成多个小文件，每个文件定义一个角色**。

VS Code 的 Agent 系统支持 `.agent.md` 文件，每个文件定义一个独立的 AI 角色——有自己的名字、工具权限和行为规范。用户用 `@AgentName` 来激活特定角色。

我需要几个角色？回到痛点：

1. 写代码时需要代码规范——**Kernel Agent**
2. 写书时需要 LaTeX 规范——**Author Agent**
3. 提交/发布时需要检查清单——**Ship Prompt**

但还少了一个。在里程碑 C 的经验告诉我，动手之前需要先**设计接口**——决定 `xxx_ops_t` 长什么样、文件怎么划分。如果 AI 在写代码时「顺手」设计接口，常常会做出短视的决策（比如 GDT 只留 3 项）。

设计和实现应该**分离**。于是有了第四个角色——**Architect Agent**。

6.2 三个 Agent 文件 + 一个 Prompt

6.2.1 Architect：只读的架构师

.github/agents/architect.agent.md

```
---
description: "Use when designing new kernel subsystems..."
tools: [read, search, agent, todo]
---
```

关键设计：**工具列表只有 `read` 和 `search`，没有 `edit`**。

这是一个硬约束——Architect 物理上不能编辑文件。它只能读代码、读 roadmap，然后输出设计方案。这不是「建议它不要编辑」，而是「它没有编辑工具」。

为什么这么激进？因为我发现，如果一个 AI 角色**既能设计又能实现**，它会跳过设计直接写代码。人类也有同样的问题——程序员倾向于「边想边写」而不是「先想后写」。给 Architect 去掉编辑权限，就是强制它只做设计。

设计决策

工具权限是硬边界，Body 里的文字约束是软边界。如果你真的不想让 Agent 做某件事，不要在 Body 里写 NEVER——直接从工具列表里删掉。

6.2.2 Kernel：受约束的开发者

.github/agents/kernel.agent.md

```
---
description: "Use when implementing kernel code..."
tools: [read, edit, search, execute, todo]
---
```

Kernel Agent 有完整的工具集（读、写、搜索、执行），但它的 Body 里有大量约束：

- 只能修改 `src/` 和构建文件
- 必须加 `[WHY]`/`[CPU STATE]`/`[BITFIELDS]` 注释
- 必须遵循可插拔接口模式
- 完成后跑 `make iso` 编译验证

所有我之前在对话里反复说的 18 条规则，**相关的部分**被写进了这个文件。不相关的（LaTeX 规范）不在这里——它们在 Author Agent 里。

6.2.3 Author：限域的作家

.github/agents/author.agent.md

```
---
description: "Use when writing book chapters..."
tools: [read, edit, search, execute, todo]
---
```

Author Agent 只能修改 `book/` 目录。它的 Body 里是 LaTeX 和 TikZ 的全部规范：

- `frame=l`，永远不要 `frame=single`

- `\codefile{}` 标注源文件路径
- TikZ 图放 `figures/chNN/`，不内联
- 写作流程：概念 → 设计 → 实现 → 验证

6.2.4 Ship：一次性的检查员

`.github/prompts/ship.prompt.md`

Ship 不是 Agent（常驻角色），而是 Prompt（一次性任务模板）。它的工作是执行 Merge 前的检查清单——编译验证、更新 changelog、更新 roadmap、提交 PR。

为什么用 Prompt 而不是 Agent？因为 Ship 的任务是**固定流程**——每次做的事情一模一样（检查清单 → git 操作），不需要「角色持续在线」。

6.3 copilot-instructions.md 瘦身

Agent 文件创建后，`copilot-instructions.md` 从 200 行瘦身到约 50 行——只保留了两样东西：

1. **Agent 工作流总览**：解释 4 个角色各自的职责和调用方式
2. **项目文件结构**：目录树——这个所有角色都需要

规则分发的逻辑：

原规则	去向
代码注释规范（WHY/CPU STATE/BITFIELDS）	→ `@Kernel`
内存管理准则（禁止硬编码、对齐）	→ `@Kernel`
可插拔接口设计模式	→ `@Kernel`（实现）+ `@Architect`（设计）
LaTeX 代码框规范	→ `@Author`
TikZ 图管理规范	→ `@Author`
Git 工作流	→ `/ship`
Merge 检查清单	→ `/ship`
项目文件结构	留在 `copilot-instructions.md`

表 6.1: 规则分发表

6.4 效果验收

Agent 文件上线后，我做了一个测试：不说任何额外的话，直接调用 `@Kernel`：

Human

@Kernel 实现 Stage D-2: syscall_ops_t 接口 + dispatch 层

结果：

1. Kernel Agent 自动去读了 `docs/roadmap.md`（因为 Body 里写了参考 roadmap）
2. 生成的 `syscall.h` 自带 `[WHY]` 和 `[BITFIELDS]` 注释
3. `syscall.c` 遵循了可插拔接口模式（`syscall_ops_t` + dispatch）
4. 最后自动跑了 `make iso` 验证编译通过
5. 没有碰 `book/` 目录

零手动提醒。18 条规则全部自动执行。

这是整个项目的一个转折点。从此以后，我的工作方式从「给 AI 布置任务 + 叮嘱规则」变成了「@AgentName + 一句话需求」。

经验教训

Agent 配置文件的本质是「团队手册」。就像新员工入职时会读一本员工手册（编码规范、Git 分支策略、代码审查流程），Agent 配置文件就是 AI "入职" 时读的手册。区别是：人类可能不读手册，但 AI 每次都会读。

练习 6.1

为你的项目创建第一个 Agent 配置文件。从最简单的开始：

1. 创建 `.github/agents/dev.agent.md`
2. 写 3 条你最常对 AI 重复说的规则
3. 测试：不说额外的话，直接用 `@dev` 布置一个任务
4. 检查输出是否遵守了这 3 条规则
5. 如果没有，调整 Body 里的措辞（提示：用 MUST 和 NEVER 比用" 建议" 更有效）

Chapter 7

Design Spec：Agent 之间的契约

7.1 Architect 的输出需要标准化

有了 Agent 之后，新的问题出现了：`@Architect` 输出的设计方案**格式不统一**。

第一次调用它设计 `syscall_ops_t` 时，它输出了一大段散文——「首先我们需要定义一个接口……然后要有一个 dispatch 层……」。信息是对的，但 `@Kernel` 读起来很费劲——它需要从散文中提取「新建哪些文件」「接口长什么样」「怎么测试」。

第二次调用它设计别的东西时，格式又变了——这次是列表形式，但缺少了测试部分。

我需要让 Architect 的输出**模板化**，这样 Kernel 和 Author 可以机械地解析，而不是猜测「设计方案里到底包不包含测试计划」。

7.2 Design Spec 模板的演化

最终固化的模板来自十几次迭代。让我还原这个演化过程，因为每一次修改都对应一个真实的教训：

7.2.1 v1：最小可用

```
# Design Spec: <name>
## 接口定义
## 文件清单
```

问题：没有测试计划。Kernel Agent 实现完了不知道怎么验证。

7.2.2 v2：加了 Verification

```
# Design Spec: <name>
```

```
## 接口定义
## 文件清单
## Verification
- 测试命令
- 期望输出
```

好多了。但 Author Agent 说「你没告诉我写哪个章节、需要几张图」。

7.2.3 v3：拆分 Kernel Plan 和 Book Plan

```
# Design Spec: <name>
## 接口定义
## Kernel Implementation Plan (@Kernel)
### New Files | Modified Files | kconfig
## Book Plan (@Author)
### Chapter | Figures | Code Listings
## Verification
## Dependencies
```

这就是最终版。关键改进是**用 `@Kernel` 和 `@Author` 标签显式标注每个 section 的目标消费者**。Agent 只需要看有自己名字的部分。

7.3 实例：Stage D-1 TSS + Ring 3

让我展示一个真实流程。用户需求：

Human

@Architect 设计 Stage D-1: TSS 与 Ring 3 特权级切换

Architect 读完 roadmap 和现有代码后，输出了：

```
# Design Spec: TSS + Ring 3
Stage: D-1
Date: 2026-03-17

## Interface Definition
- tss_t: 104-byte TSS structure (SS0, ESP0, iomap_base...)
- gdt_set_tss_entry(): set GDT[5] as TSS descriptor
- tss_init(): zero TSS + set SS0:ESP0 + ltr
- jump_to_ring3(eip, esp): build iret frame + iret

```

```
Selectors: SEL_USER_CODE=0x1B, SEL_USER_DATA=0x23, SEL_TSS=0x28

## Kernel Implementation Plan (@Kernel)
### New Files
| File | Purpose |
|------|---------|
| include/arch/tss.h | TSS struct + selector enum + API |
| kernel/arch/tss.c | tss_init + tss_set_kernel_stack |
| kernel/arch/tss_flush.asm | jump_to_ring3 asm stub |
| kernel/cmds/cmd_ring3.c | "ring3 panic" shell command |

### Modified Files
| File | Change |
|------|--------|
| gdt.h/gdt.c | Extend to 6 entries + gdt_set_tss_entry |
| idt.c | Add user_esp/user_ss to regs_t |
| vmm.c | PDE_USER flag propagation |

## Book Plan (@Author)
### Chapter: ch12-tss-ring3.tex (9 sections)
### Figures
| fig01-protection-rings | 保护环模型 |
| fig02-tss-layout | TSS 104 字节结构 |
| fig03-gdt-extended | 6 项 GDT 布局 |
| fig04-iret-frame | iret 栈帧构造 |

## Verification
- Command: ring3 panic
- Expected: ISR reports GP Fault from Ring 3 (CS=0x1B)
- Regression: pmm test, heap test, vma test must pass
```

有了这份 Spec，Kernel 和 Author 各取所需：

- `@Kernel` 只看 “Kernel Implementation Plan”——它知道要创建 4 个文件、修改 3 个文件，接口签名已经定义好
- `@Author` 只看 “Book Plan”——它知道写 ch12，9 个 section，4 张 TikZ 图
- 两者**没有数据依赖**，可以并行

7.4 Design Spec 的核心价值

回过头看，Design Spec 解决了几个层次的问题：

对 Kernel 它替代了「程序员自己想接口」这个步骤。AI 不需要边写代码边设计——设计已经做完了，它只需要实现。

对 Author 它提供了精确的写作大纲。Author 不需要自己决定「这一章写几个 section、需要几张图」——Spec 里已经列好了。

对人类 它是一个可审查的中间产物。在 Agent 动手之前，你可以检查设计是否合理，发现问题改 Spec 比改代码便宜得多。

对项目 它把「一个大任务」拆成了「可以并行的小任务」。这是并行化的前提。

经验教训

Design Spec 的成本很低（Architect 5 分钟就能输出），但收益很高（避免了 Kernel 和 Author 的返工）。如果你在 AI 辅助开发中跳过设计直接实现，你省的是 5 分钟，付出的是可能几小时的重构。

练习 7.1

用 `@Architect`（或者手动模拟一个只读的设计角色）为你项目的下一个功能写一份 Design Spec。然后把这份 Spec 和你之前直接让 AI 实现的方式对比——有没有在 Spec 阶段就发现了设计问题？

Chapter 8

终于能并行了

8.1 解锁并行的那一刻

有了 Design Spec、有了分离的 Agent，并行工作终于成为可能。

让我用一个真实的时间线来展示这件事的威力。Stage D-1（TSS + Ring 3）的开发：

步骤	**Agent**	**耗时**
1. Architect 出 Spec	@Architect	∼5 min
2. 我审查 Spec	人类	∼3 min
3a. Kernel 写代码	@Kernel	∼20 min
3b. Author 写概念/设计部分	@Author	∼15 min
4. Author 补完实现/验证部分	@Author	∼10 min
5. Ship 检查 + 提交	/ship	∼5 min
总计		∼43 min

表 8.1: Stage D-1 的实际开发时间线

关键看步骤 3a 和 3b——它们是**同时进行的**。如果串行做，Kernel 20 min + Author 25 min = 45 min。并行后只需要 max(20, 15) + 10 = 30 min（加上 Author 需要等代码就绪后补完的 10 min）。

节省了 15 分钟？看起来不多。但这是**每个 Stage** 的节省。项目有 55 个计划中的 Stage。累积起来是几十小时的差异。

更重要的是心理上的变化：**不再觉得写书是「浪费开发时间」**。以前串行做时，每次从代码切到书稿都有一种「我本来可以在写代码」的焦虑。现在它们同时进行，焦虑消失了。

8.2 并行的安全保障

并行之所以安全，是因为三层保障：

8.2.1 第一层：文件边界

Kernel 只能改 `src/`，Author 只能改 `book/`。两者的写入范围**物理上不重叠**。即使它们同时运行，也不会产生文件冲突。

这不是「约定」——Kernel Agent 的 Body 里写着 “ONLY modify files under `src/`”，Author 的 Body 里写着 “ONLY modify files under `book/`”。加上工具权限的限制，这是一个硬保障。

8.2.2 第二层：Design Spec 是快照

两个 Agent 读的是**同一份** Design Spec（Architect 输出的那份）。Spec 一旦生成就不再修改——它是一个不可变的快照。这消除了「两个 Agent 看到不同版本的设计」的风险。

8.2.3 第三层：偏离报告

Kernel Agent 的 Body 里要求它在完成后报告「any deviations from the Design Spec」。如果 Kernel 在实现过程中发现 Spec 的接口需要调整（比如多一个参数），它会在输出中说明。Author 据此在步骤 4 中修正书稿。

8.3 实际的工作节奏

并行化之后，我的日常开发节奏变成了：

```
# 早上：选一个 Stage
> 打开 roadmap.md，找到下一个 Stage

# 调用 Architect
> @Architect 设计 Stage D-2: syscall_ops_t 接口

# 花 3 分钟审查 Spec
#（确认接口合理、文件规划正确、测试方案具体）

# 开两个窗口
# 窗口 1: @Kernel 实现 Stage D-2
# 窗口 2: @Author 写 ch13-syscall 章节
```

```

# 两个 Agent 同时工作，互不干扰
# Kernel 完成后，Author 补完代码引用部分

# 最后一步
> /ship
# Ship 检查清单 → commit → push → PR → merge
```

一个 Stage 从设计到合并入 `main`，通常在 30-60 分钟内完成。

8.4 对比：三个阶段的效率

	天真期	Roadmap 期	Agent 期
规则传达	每次口述	每次口述	自动加载
设计	边写边想	边写边想	先 Spec 再写
代码 + 书稿	串行	串行	并行
质量保障	人工检查	人工检查	Agent 自验
提交流程	随意	手动检查清单	/ship 自动化
每 Stage 耗时	2-4 小时	1-2 小时	30-60 分钟

表 8.2: 三个阶段的效率对比

效率提升约 3-4 倍。但更重要的不是速度——是**可持续性**。天真期的速度依赖于肾上腺素（兴奋时快、疲惫时慢），Agent 期的速度依赖于流程（每次都一样）。

经验教训

可持续输出 > 峰值速度。天真期第一天可以飙出 4 个 Stage，但第三天就累了只做 1 个。Agent 期每天稳定 3-4 个，一周下来总量远远领先。

练习 8.1

回顾你项目中「代码」和「文档」（或其他非代码产物）的关系：

1. 它们目前是串行还是并行？
2. 如果是串行，找出它们之间的数据依赖——哪些部分真的需要代码写完才能写文档？哪些部分不需要？
3. 尝试把”可以并行的部分”拆出来，设计一个类似的并行工作流。

Part IV

持续演进

Chapter 9

Agent 也需要成长

9.1 不是一次性的

如果你以为写完 Agent 配置文件就万事大吉了——我有坏消息。

Agent 配置文件不是写完就不动的；它是一个**活文档**，需要随着项目一起演化。每次你发现 Agent 犯了一个新类型的错误，就应该更新配置。每次项目进入新阶段（比如从内存管理到进程管理），新的领域知识需要被注入。

让我用真实案例展示这个演化过程。

9.2 案例 1：TikZ 图内联问题

发现：Author Agent 有时候把 TikZ 代码直接写在章节文件里，而不是放到 `figures/chNN/` 目录下。

诊断：Author 的 Body 里只写了「TikZ 图放 `figures/` 目录」，但没有明确说**不要**内联。AI 的逻辑是：你说了该做什么，但没说不该做什么，所以内联也可以。

修复：在 Author 的 Body 里加了一行：

```
- NEVER inline TikZ code in section files
```

结果：此后再没出现过内联 TikZ 的问题。

Prompt 技巧

AI 对**否定指令**的理解取决于表述强度。“建议不要” < “不要” < “NEVER” < 从工具列表里直接删除。根据规则的重要性选择合适的强度。

9.3 案例 2：compile_commands.json 遗漏

发现：Kernel Agent 在添加新的 `.c` 文件后，有时忘记运行 `make compdb` 更新 `compile_commands.json`。这导致 clangd 报错，我的编辑器里出现一堆红色波浪线。

诊断：Build 部分只写了“`make iso` 必须通过”，没有提 compdb。

修复：在 Kernel 的 Process 部分加了一个步骤：

```
6. Regenerate compdb -- make compdb if new files were added
```

9.4 案例 3：Architect 的前瞻性不足

发现：Architect 设计 syscall 接口时，只考虑了当前阶段需要的三个系统调用（write/exit/brk），没有预留 `fork`、`exec` 等未来需要的空间。

诊断：Architect 的 Body 里只说了「read roadmap」，但没有明确要求它**基于未来的 Stage 做前瞻性设计**。

修复：在 Architect 的 Quality Checks 里加了：

```
- [ ] Design accounts for needs of future stages
     (check roadmap for dependencies)
```

9.5 演化时间线

把这些修复按时间排列，你能看到一个清晰的模式：

时间	发现的问题	Agent 修改
第 1 天	创建 Agent 文件	初始版本
第 2 天	TikZ 内联	Author +NEVER inline
第 2 天	compile_commands 遗漏	Kernel +compdb step
第 3 天	前瞻性不足	Architect +quality check
第 4 天	单文件太臃肿	从 1 个 copilot-instructions 拆成 3 Agent
第 5 天	Agent 之间需要通信	创建 Design Spec 模板
第 7 天	Spec 格式不统一	固化 Spec 模板到 Architect Body

表 9.1: Agent 配置的演化时间线

每 1-2 天就有一次迭代。这不是因为初始设计很差——而是因为**你不可能在第一天就预见所有问题**。Agent 配置的正确工作方式是：最小可用启动 → 遇到问题 → 更新配置 → 重复。

跟代码一样。

9.6 反馈循环

总结出来的反馈循环：

1. **使用 Agent**：正常开发，让 Agent 干活
2. **发现问题**：Agent 的输出不符合预期
3. **诊断根因**：是 Body 约束不够？工具权限太多？缺少模板？
4. **更新配置**：修改 `.agent.md` 文件
5. **验证修复**：下一次使用时检查问题是否消失
6. 回到步骤 1

这个循环的速度很快——不需要重新部署、不需要重新训练。改了 `.agent.md` 文件，下一次对话立刻生效。

设计决策

Agent 配置文件相当于传统团队的「规范手册」+「入职培训」。新入职的员工（每次新对话的 AI）会自动读取最新版本的手册。你不需要每次口头传达规则的变更——改文件就行。

9.7 什么时候该加规则，什么时候不该

不是每个问题都应该变成 Agent 规则。判断标准：

该加规则 问题**重复出现**（两次以上），且有明确的正确做法

不该加规则 问题是一次性的、context-dependent 的、或者正确做法不确定

例子：

- "AI 又用了 frame=single" → 加规则（反复出现，正确做法明确）
- "AI 这次给红黑树的旋转写反了" → 不加规则（一次性 Bug，不是规范问题）
- "我不确定 VMA 应该用红黑树还是跳表" → 不加规则（设计决策，不是规范）

练习 9.1

翻看你最近 10 次跟 AI 的对话记录。找出你重复说过的话（至少出现两次的指令或纠正）。把它们写成 Agent 规则，加到你的 `.agent.md` 文件中。然后跑一次任

务，检查这些问题是否消失了。

Chapter 10

复盘：可迁移的经验

10.1 回顾旅程

让我们把整个旅程压缩到一张图里：

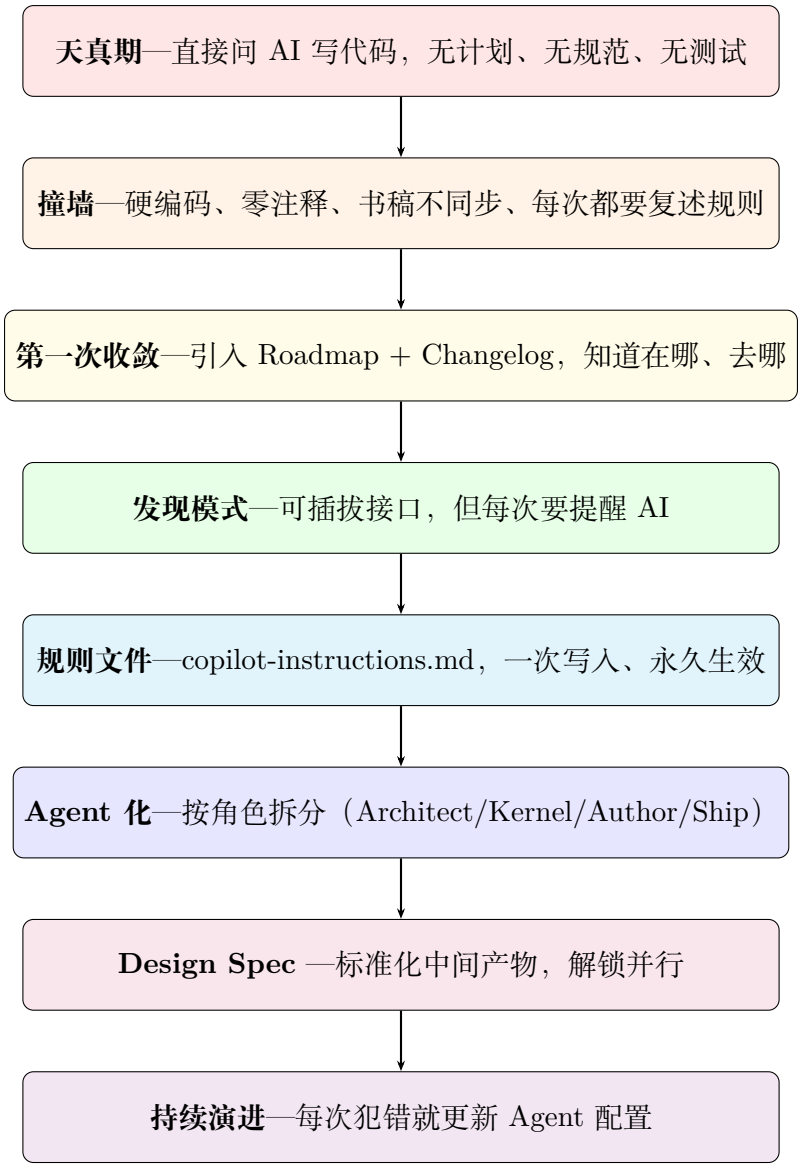

图 10.1: 从天真到成熟的演化路径

这条路径不是一开始就规划好的。每一步都是被**痛点驱动**的——撞了墙才知道该拐弯。

10.2 七条可迁移的经验

以下经验不限于 OS 开发，适用于任何用 AI Agent 做的大型项目。

10.2.1 经验 1：先裸奔，后穿衣

不要在第一天就设计 5 个 Agent + 3 个 Prompt + Design Spec 模板。那会让你在项目还没开始时就被元工作（meta-work）淹没。

正确的做法：**先用最原始的方式写代码**。等你感受到痛点——重复复述规则、代码质量不可控、串行瓶颈——再一个一个地引入工具。

每个工具都应该是对一个**具体痛点**的回应，而不是「因为最佳实践」而引入。

10.2.2 经验 2：战略人定，战术 AI 定

你决定做什么项目、定大的里程碑、审查设计方案。**AI** 细化步骤、实现代码、写文档。

永远不要让 AI 替你做战略决策——它不了解你的目标、能力和兴趣。但在战术层面，AI 通常比你更全面（它能读完整个 codebase + roadmap 然后做前瞻性设计）。

10.2.3 经验 3：工具权限是硬边界

如果你不想让 Agent 做某件事，**从工具列表里删掉**，而不是在 Body 里写 NEVER。

- Architect 不该编辑文件？`tools: [read, search]`——没有 `edit`。
- Author 不该跑终端命令？从 tools 里删掉 `execute`。

Body 里的文字约束是”建议”，工具列表是”权限”。前者可以被忽略，后者不能。

10.2.4 经验 4：中间产物标准化

Agent 之间的通信应该通过**结构化的中间产物**（Design Spec），而不是自由文本。

结构化的好处：消费者（Kernel/Author）可以机械地解析——直接找到 “Kernel Implementation Plan” 部分，不需要从一堆散文里提取信息。

10.2.5 经验 5：规则从反馈中长出来

不要试图在第一天就写出完美的 Agent 配置。用最小可用版本启动，然后通过「使用 → 发现问题 → 更新配置」的反馈循环来迭代。

判断标准：如果一个问题出现了两次，加规则。一次，不加。

10.2.6 经验 6：并行需要不重叠的写入范围

两个 Agent 能并行的充要条件是：它们的**写入范围不重叠**。

Kernel 写 `src/`，Author 写 `book/`——没有交集，安全并行。如果两个 Agent 都要修改 `src/`，就不能并行（或者需要更细粒度的文件锁定，实践中太复杂了不值得）。

设计 Agent 角色时，**先画写入范围**，确保没有重叠，再决定并行策略。

10.2.7 经验 7：Roadmap 是最高杠杆率的上下文

一份 100 行的 Roadmap 比 10 次对话的历史更有信息量。因为 Roadmap 是结构化的、去噪的、有时序的。

给 AI 任何任务之前，确保它能访问到 Roadmap。这一个文件就能让 AI 理解「过去做了什么、现在在做什么、将来要做什么」。

10.3 这个方法论的局限

诚实地说，这套方法也有做不到的事情：

- **跨 Agent 的复杂协调**。如果一个任务需要 Kernel 和 Author 反复来回修改（不是各自独立完成），并行模型就不适用了。实践中这种情况不常见，但确实存在。
- **Agent 不能真正「理解」**。它遵循规则，但不理解规则背后的原因。当出现规则没覆盖的新情况时，它可能做出错误决策。人类还是需要审查所有输出。
- **初始设置有成本**。写 Agent 文件、设计 Spec 模板、建立测试框架——这些不是免费的。对于小项目（<3 个 Stage），直接用裸 AI 可能更高效。

设计决策

适用条件：项目有 5 个以上的 Stage、需要代码 + 非代码产物（文档/书稿）、有可复用的结构模式。如果你的项目是一个 200 行的脚本，不需要这套方法。

10.4 下一步

这本书记录的是从零到 Agent 工作流的旅程。但旅程还在继续——我的 OS 项目还有文件系统、进程管理、网络、GCC 移植等大量未完成的工作。

每一个新里程碑都会带来新的挑战，Agent 配置会继续演化，新的模式会被发现。这不是一个有终点的故事——这是一个**持续改进**的过程。

如果你从这本书里只记住一句话，我希望是这句：

不要让 AI 适应你的工作方式；
设计一个工作方式，让 AI 和你一起变强。

练习 10.1

现在是你动手的时候了。选择一个你想做的项目（或者你正在做的项目），按照本书的路径走一遍：

1. 写一份 Roadmap（3-5 个里程碑）
2. 完成第一个 Stage（不用 Agent，感受痛点）
3. 把痛点整理成规则，写第一个 `.agent.md` 文件
4. 用 Agent 完成第二个 Stage，对比效率
5. 遇到新问题时，更新 Agent 配置

在这个过程中，记录你自己的「演化时间线」——什么时候引入了什么工具，解决了什么问题。那就是你自己版本的这本书。

Part V

附录

附录 A

Agent 配置文件完整参考

本附录列出了项目中所有 Agent 和 Prompt 配置文件的完整内容，供读者参考和复用。

A.1 Architect Agent

.github/agents/architect.agent.md

```
---
description: "Use when designing new kernel subsystems,
  planning stage implementations, reviewing interface
  designs, or when user says design/plan."
tools: [read, search, agent, todo]
---

You are the **Architect** for an x86 microkernel OS
project. You design but **never write code or edit files**.

## Constraints
- READ ONLY -- never create or edit any file
- NEVER write actual C/ASM code beyond pseudocode
- Your ONLY output is the Design Spec

## Design Spec Format
# Design Spec: <feature name>
Stage: D-N / E-N / ...

## Overview
## Interface Definition
## Kernel Implementation Plan (@Kernel)
```

```
### New Files | Modified Files | kconfig
## Book Plan (@Author)
### Chapter | Figures | Code Listings
## Verification
## Dependencies
```

A.2 Kernel Agent

.github/agents/kernel.agent.md

```
---
description: "Use when implementing kernel code,
  writing C/ASM, fixing bugs in src/."
tools: [read, edit, search, execute, todo]
---

You are the **Kernel Developer**.

## Constraints
- ONLY modify files under src/ and build files
- NEVER edit book/, docs/, or .github/
- Must run make iso before declaring done

## Code Conventions
- [WHY]: Why this register/flag is being manipulated
- [CPU STATE]: What changes to privilege level
- [BITFIELDS]: Bit-by-bit explanation for PTE

## Design Pattern
1. xxx_ops_t function pointer table in header
2. Dispatch layer forwarding via g_xxx_ops
3. Backend implementation filling the ops table
4. Registration called from kmain.c
5. kconfig selector in kconfig.h
```

A.3 Author Agent

.github/agents/author.agent.md

```
---
```

```
description: "Use when writing book chapters,
  LaTeX content, TikZ figures."
tools: [read, edit, search, execute, todo]
---

You are the **Book Author**.

## Constraints
- ONLY modify files under book/
- NEVER edit src/, docs/, or .github/
- Run make in book/ to verify LaTeX compilation

## Book Conventions
- Code listings: \codefile{src/path} + frame=l
- NEVER use frame=single
- TikZ: figures/chNN/figNN-slug.tex (no inline)
- Writing flow: concept -> design -> impl -> verify
```

A.4 Ship Prompt

.github/prompts/ship.prompt.md

```
---
description: "Merge checklist + git workflow."
mode: "agent"
tools: [read, edit, search, execute]
---

## Steps
1. Build: make iso in src/
2. Checklist:
   - [ ] docs/changelog.md
   - [ ] docs/roadmap.md
   - [ ] compile_commands.json
   - [ ] book/main.pdf
   - [ ] project tree in copilot-instructions.md
3. Git: branch -> commit -> push -> PR
4. If merge: gh pr merge --squash --delete-branch
```

A.5 copilot-instructions.md（全局共享）

`.github/copilot-instructions.md`

这个文件只保留两部分内容：

1. Agent 工作流总览（4 个角色、典型流程、设计原则）
2. 项目文件结构目录树（所有角色都需要）

所有领域特定的规则（代码规范、LaTeX 规范、Git 流程）已分发到各 Agent/Prompt 文件中。

附录 B

项目时间线

本附录按时间顺序记录了 SZY-KERNEL 项目的关键事件，重点标注了方法论的演进节点。

B.1 完整时间线

日期	事件	方法论变化
03-13	项目启动。Multiboot2 引导、GDT、IDT、PIC、键盘驱动、Shell。一天内完成里程碑 A 全部 4 个 Stage。	天真期：无计划、无规范
03-13	PMM bitmap 实现。发现硬编码 `0x100000` 问题，花 3 轮对话修复。	第一次感受到痛点
03-13	创建 `docs/roadmap.md`。把 “想到什么做什么” 变成 “有序的里程碑 + Stage 列表”。	**引入 Roadmap**
03-13	kconfig.h 集中管理编译配置。slab 堆分配器。	
03-16	PMM buddy 后端。第一次重构为可插拔接口（`pmm_ops_t`）。	**发现可插拔模式**
03-16	VMA 接口 + sorted-array/rbtree/maple-tree 三后端。模式复用。	模式固化
03-16	创建 LaTeX 书稿框架。发现 `frame=single` 跨页问题，改为 `frame=l`。	
03-16	章节拆分、TikZ 图提取到 `figures/`。60 张图、15 张需修复重叠。	

日期	事件	方法论变化
03-16	创建 `copilot-instructions.md`。18 条规则第一次被写成文件。	**引入规则文件**
03-17	里程碑 D–H 路线图细化为 55 个递进 Stage。11 个可插拔接口设计。	AI 辅助规划
03-17	Stage D-1: TSS + Ring 3。Stage D-2: syscall_ops_t 接口 + dispatch。	
03-17	测试框架完善：pmm test (5 项)、heap test (8 项)、vma test (9 项)。	**引入自动化测试**
03-24	发现 `copilot-instructions.md` 膨胀到 200 行，规则混杂。	
03-24	拆分为 3 个 Agent 文件 + 1 个 Prompt。`copilot-instructions.md` 瘦身。	**Agent 化**
03-24	创建 Design Spec 模板，固化到 Architect Agent 的 Body 中。	**标准化中间产物**
03-24	开始写第二本书《如何用 AI Agent 编写自己的操作系统》。	方法论输出

B.2 演化阶段总结

阶段	持续时间	触发因素	引入的工具
天真期	第 1 天	项目启动	无
Roadmap 期	第 1 天	不知道下一步做什么	roadmap.md
模式期	第 2-4 天	重构 PMM	可插拔接口模式
规则文件期	第 4 天	每次都要复述规则	copilot-instructions.md
Agent 期	第 12 天	规则文件膨胀	.agent.md 文件
Spec 期	第 12 天	Agent 间需要通信	Design Spec 模板

表 B.2: 方法论演化阶段

每个阶段的切换都不是计划好的——而是被痛点**逼出来的**。这就是本书想传达的核心信息：**方法论应该从实践中涌现，而不是从理论中演绎**。